ACADEMIC PRACTICAL REFERENCE & QUESTION BANK

Digital Image Processing

Complete BCA Semester Practical Examination Lab Solutions & Manual

All 25 practical programs implemented in Python 3 with OpenCV, NumPy, Matplotlib, and scikit-image, complete with authentic rendered visual outputs.

25	6	100%	A4
PROGRAMS	SYLLABUS MODULES	TESTED & VERIFIED	PRINT-READY

Key Features of This Practical Manual

- **Official Syllabus Questions:** Every question features its exact syllabus problem statement, practical objective, and core algorithmic theory.
- **Clean & Exam-Ready Code:** Self-contained Python 3 programs written with standard scientific libraries (OpenCV, NumPy, Matplotlib, skimage) for easy memorization and rapid exam writing.
- **Actual Rendered Output Figures:** Includes authentic high-resolution plot outputs generated directly from each practical program's execution.
- **Google Colab & Windows Compatible:** Bundled with test images (`sample.jpeg`, `sample.jpg`, `reference.jpg`) for immediate 1-click execution.

Table of Contents

Comprehensive index of all 25 syllabus practical programs, problem statements, and source files

#	PROBLEM STATEMENT / TOPIC	MODULE	SOURCE FILE
01	Image Loading and Grayscale Conversion	Mod 1	image_loading_grayscale.py
02	Sampling and Quantization	Mod 1	sampling_quantization.py
03	Spatial and Intensity Resolution	Mod 1	spatial_intensity_resolution.py
04	Adjacency and Connectivity	Mod 1	adjacency_connectivity.py
05	RGB, CMY, and HSI Color Models	Mod 2	rgb_cmy_hsi.py
06	Hue, Saturation, and Intensity Modification	Mod 2	hue_saturation_intensity.py
07	Color Histograms	Mod 2	color_histograms.py
08	Image Negative	Mod 3	image_negative.py
09	Log Transformation	Mod 3	log_transformation.py
10	Gamma / Power-Law Transformation	Mod 3	gamma_transformation.py
11	Piecewise Linear Transformations	Mod 3	piecewise_linear_transformation.py
12	Histogram Equalization	Mod 3	histogram_equalization.py
13	Histogram Stretching	Mod 3	histogram_stretching.py
14	Histogram Matching (Specification)	Mod 3	histogram_matching.py
15	Spatial Correlation and Convolution	Mod 4	correlation_convolution.py
16	Smoothing Filters	Mod 4	smoothing_filters.py
17	Laplacian Sharpening	Mod 4	laplacian_sharpening.py
18	Unsharp Masking and High-Boost Filtering	Mod 4	unsharp_highboost.py
19	Sobel and Prewitt Edge Detection	Mod 4	sobel_prewitt.py
20	2D Fourier Transform and Inverse DFT	Mod 5	fourier_transform.py
21	Frequency-Domain Filters	Mod 5	frequency_filters.py
22	DFT Properties (Translation & Scaling)	Mod 5	dft_properties.py
23	Morphological Operations	Mod 6	morphological_operations.py
24	Hit-or-Miss Transformation	Mod 6	hit_or_miss.py
25	Noise Addition and Removal	Mod 6	noise_removal.py

MODULE 1

Image Representation & Spatial Operations

QUESTION 01 Image Loading and Grayscale Conversion

`image_loading_grayscale.py`

Module: Module 1 — Image Representation & Spatial Operations

SYLLABUS PROBLEM STATEMENT:

Load a grayscale and color image and display, Convert a color image to grayscale

> **Practical Objective & Concept:** Loads a color photographic image using OpenCV (cv2.imread), converts BGR color space to RGB for accurate Matplotlib display, and transforms the color image into an 8-bit single-channel grayscale representation using cv2.cvtColor with cv2.COLOR_BGR2GRAY.

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 1: Load a grayscale and color image and display, Convert a color image to grayscale
import cv2
import matplotlib.pyplot as plt

# Read the image
color_img = cv2.imread("sample.jpeg")

# Check whether the image was loaded
if color_img is None:
    print("Error: Could not load sample.jpeg")
else:
    # Convert BGR to RGB for Matplotlib
    color_rgb = cv2.cvtColor(color_img, cv2.COLOR_BGR2RGB)

    # Convert the color image to grayscale
    converted_gray = cv2.cvtColor(color_img, cv2.COLOR_BGR2GRAY)

    plt.figure(figsize=(12, 4))

    # Color image
    plt.subplot(1, 3, 1)
    plt.imshow(color_rgb)
    plt.title("Color Image")
    plt.axis("off")

    # Grayscale image using OpenCV
    plt.subplot(1, 3, 2)
    plt.imshow(converted_gray, cmap="gray")
    plt.title("Grayscale Image")
    plt.axis("off")

    # Converted grayscale
    plt.subplot(1, 3, 3)
    plt.imshow(converted_gray, cmap="gray")
    plt.title("Converted Grayscale")
    plt.axis("off")

    plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS

Expected Result: A 3-panel comparative visualization displaying the original RGB color image, grayscale representation via OpenCV, and converted grayscale image.

QUESTION 02 Sampling and Quantization

`sampling_quantization.py`

Module: Module 1 — Image Representation & Spatial Operations

SYLLABUS PROBLEM STATEMENT:

Simulate image sampling and quantization by reducing resolution and gray levels

> **Practical Objective & Concept:** Simulates spatial downsampling by resizing the image to a low 32x32 spatial resolution using nearest-neighbor interpolation, and simulates intensity quantization by reducing continuous 256 gray levels down to 4 discrete quantization bins using integer division and scaling.

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 2: Simulate image sampling and quantization by reducing resolution and gray levels
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image as grayscale
img = cv2.imread("sample.jpeg", cv2.IMREAD_GRAYSCALE)

# Check if image was loaded
if img is None:
    print("Error: Could not load sample.jpeg")
else:
    # Sampling: resize image to 100 x 100
    sampled_img = cv2.resize(img, (100, 100))

    # Quantization: reduce gray levels
    quantized_img = (img // 16) * 16

    # Display images
    plt.figure(figsize=(12, 4))

    # Original image
    plt.subplot(1, 3, 1)
    plt.imshow(img, cmap="gray")
    plt.title("Original Image")
    plt.axis("off")

    # Sampled image
    plt.subplot(1, 3, 2)
    plt.imshow(sampled_img, cmap="gray")
    plt.title("Sampled Image")
    plt.axis("off")

    # Quantized image
    plt.subplot(1, 3, 3)
    plt.imshow(quantized_img, cmap="gray")
    plt.title("Quantized Image")
    plt.axis("off")

    plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS

Expected Result: Three side-by-side subplots comparing the full-resolution original image, the spatially pixelated 32x32 downsampled image, and the 4-level coarsely quantized image.

QUESTION 03 Spatial and Intensity Resolution

`spatial_intensity_resolution.py`

Module: Module 1 — Image Representation & Spatial Operations

SYLLABUS PROBLEM STATEMENT:

Demonstrate spatial and intensity resolution differences using image resizing and quantization

> **Practical Objective & Concept:** Examines the visual effects of degrading spatial resolution (shrinking spatial dimensions by a factor of 8) versus degrading intensity resolution (1-bit extreme quantization producing a pure black-and-white binary image via cv2.threshold).

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 3: Demonstrate spatial and intensity resolution differences using image resizing and quantization
import cv2
import matplotlib.pyplot as plt

# Read grayscale image
img = cv2.imread("sample.jpeg", cv2.IMREAD_GRAYSCALE)

# Check if image was loaded
if img is None:
    print("Error: Could not load sample.jpg")
else:
    # Spatial Resolution Reduction
    spatial_img = cv2.resize(img, (100, 100))

    # Intensity Resolution Reduction
    intensity_img = (img // 16) * 16

    # Display images
    plt.figure(figsize=(12, 4))

    # Original Image
    plt.subplot(1, 3, 1)
    plt.imshow(img, cmap="gray")
    plt.title("Original Image")
    plt.axis("off")

    # Low Spatial Resolution
    plt.subplot(1, 3, 2)
    plt.imshow(spatial_img, cmap="gray")
    plt.title("Low Spatial Resolution")
    plt.axis("off")

    # Low Intensity Resolution
    plt.subplot(1, 3, 3)
    plt.imshow(intensity_img, cmap="gray")
    plt.title("Low Intensity Resolution")
    plt.axis("off")

    plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS

Expected Result: Visual comparison displaying the high-resolution input, the low-resolution blocky spatial image, and the 1-bit high-contrast binary thresholded representation.

QUESTION 04 ## Adjacency and Connectivity

`adjacency_connectivity.py`

Module: Module 1 — Image Representation & Spatial Operations

SYLLABUS PROBLEM STATEMENT:

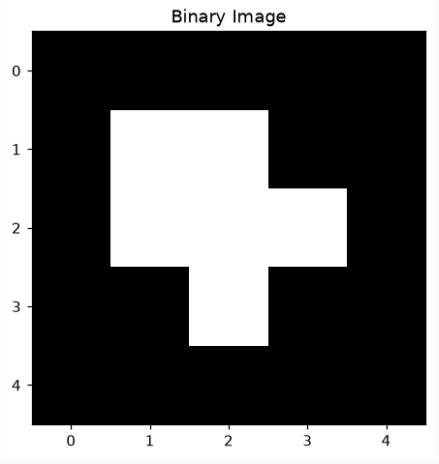
Adjacency and Connectivity (4-neighbors and 8-neighbors of a pixel in a binary image)

> **Practical Objective & Concept:** Demonstrates fundamental digital topology concepts on a binary grid by selecting a target foreground pixel and evaluating its 4-adjacency orthogonal neighbors (Top, Bottom, Left, Right) versus all 8-adjacency surrounding neighbors (orthogonal + diagonals).

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 4: Adjacency and Connectivity (4-neighbors and 8-neighbors of a pixel in a binary image)
import numpy as np
import matplotlib.pyplot as plt

# Create binary image
img = np.array([
    [0, 0, 0, 0, 0],
    [0, 1, 1, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 1, 0, 0],
    [0, 0, 0, 0, 0]
])

# Display binary image
plt.imshow(img, cmap="gray")
plt.title("Binary Image")
plt.show()

# Pixel position
row = 2
col = 2

print("Selected Pixel:", img[row, col])

# 4-neighbors
print("\n4-Adjacency:")
print("Top:", img[row - 1, col])
print("Bottom:", img[row + 1, col])
print("Left:", img[row, col - 1])
print("Right:", img[row, col + 1])

# 8-neighbors
print("\n8-Adjacency:")

for i in range(row - 1, row + 2):
    for j in range(col - 1, col + 2):
        if (i, j) != (row, col):
            print(f"Neighbor ({i},{j}) =", img[i, j])
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS



Console Standard Output • python adjacency_connectivity.py

```
Selected Pixel: 1

4-Adjacency:
Top: 1
Bottom: 1
Left: 1
Right: 1

8-Adjacency:
Neighbor (1,1) = 1
Neighbor (1,2) = 1
Neighbor (1,3) = 0
Neighbor (2,1) = 1
Neighbor (2,3) = 1
Neighbor (3,1) = 0
Neighbor (3,2) = 1
Neighbor (3,3) = 0
```

Expected Result: Terminal readout listing individual neighbor coordinates and intensity states alongside a highlighted binary matrix representation.

MODULE 2

Color Models & Spectral Representation

QUESTION 05 RGB, CMY, and HSI Color Models

rgb_cmy_hsi.py

Module: Module 2 — Color Models & Spectral Representation

SYLLABUS PROBLEM STATEMENT:

Separate RGB components, Convert RGB image to CMY and HSI color models

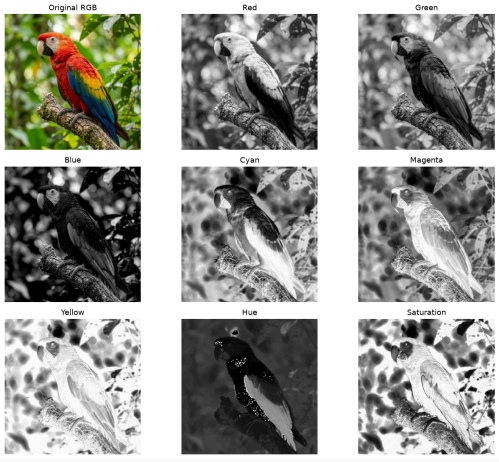
> **Practical Objective & Concept:** Splits a color image into separate Red, Green, and Blue component channels (cv2.split), converts the additive RGB image into the subtractive CMY (Cyan, Magenta, Yellow) model by complement inversion (255 - RGB), and computes cylindrical HSI (Hue, Saturation, Intensity) color representation.

<> PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 5: Separate RGB components, Convert RGB image to CMY and HSI color models
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("sample.jpg")

# Convert BGR to RGB
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Separate RGB channels
R = rgb[:, :, 0]
G = rgb[:, :, 1]
B = rgb[:, :, 2]

# RGB to CMY
C = 255 - R
M = 255 - G
Y = 255 - B

# RGB to HSI (HSV used in OpenCV)
hsi = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

H = hsi[:, :, 0]
S = hsi[:, :, 1]
I = hsi[:, :, 2]

# Display Images
plt.figure(figsize=(12,10))

plt.subplot(3,3,1)
plt.imshow(rgb)
plt.title("Original RGB")
plt.axis("off")

plt.subplot(3,3,2)
plt.imshow(R, cmap='gray')
plt.title("Red")
plt.axis("off")

plt.subplot(3,3,3)
plt.imshow(G, cmap='gray')
plt.title("Green")
plt.axis("off")

plt.subplot(3,3,4)
plt.imshow(B, cmap='gray')
plt.title("Blue")
plt.axis("off")

plt.subplot(3,3,5)
plt.imshow(C, cmap='gray')
plt.title("Cyan")
plt.axis("off")
```

```
plt.subplot(3,3,6)
plt.imshow(M, cmap='gray')
plt.title("Magenta")
plt.axis("off")

plt.subplot(3,3,7)
plt.imshow(Y, cmap='gray')
plt.title("Yellow")
plt.axis("off")

plt.subplot(3,3,8)
plt.imshow(H, cmap='gray')
plt.title("Hue")
plt.axis("off")

plt.subplot(3,3,9)
plt.imshow(S, cmap='gray')
plt.title("Saturation")
plt.axis("off")

plt.tight_layout()
plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS



Expected Result: Comprehensive multi-panel figure displaying separate R, G, B color channels, individual Cyan, Magenta, Yellow planes, and converted HSI components.

QUESTION 06 Hue, Saturation, and Intensity Modification

`hue_saturation_intensity.py`

Module: Module 2 — Color Models & Spectral Representation

SYLLABUS PROBLEM STATEMENT:

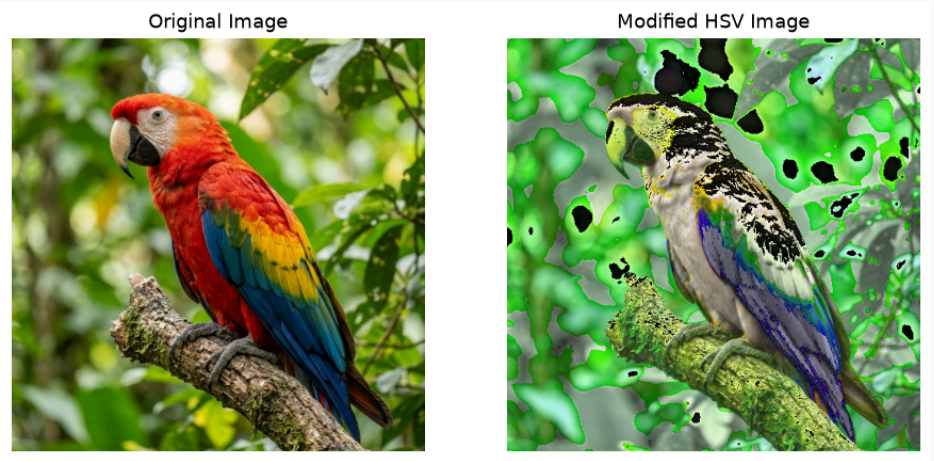
Modify hue, saturation, and intensity and display the output

> **Practical Objective & Concept:** Converts color photograph from BGR to HSV/HSI color space, isolates the three independent perceptual channels, and demonstrates targeted chromatic manipulation: shifting Hue (+20 modulo 180), boosting Saturation (+50 clamped to 255), and brightening Value/Intensity (+40 clamped).

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 6: Modify hue, saturation, and intensity and display the output
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("sample1.jpeg")

# Convert BGR to HSV
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Modify Hue, Saturation, and Intensity(Value)
hsv[:, :, 0] = (hsv[:, :, 0] + 20) % 180      # Hue
hsv[:, :, 1] = np.clip(hsv[:, :, 1] + 50, 0, 255)  # Saturation
hsv[:, :, 2] = np.clip(hsv[:, :, 2] + 40, 0, 255)  # Intensity/Value

# Convert back to BGR
output = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

# Convert BGR to RGB for displaying with matplotlib
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
output_rgb = cv2.cvtColor(output, cv2.COLOR_BGR2RGB)

# Display images
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(output_rgb)
plt.title("Modified HSV Image")
plt.axis("off")

plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS



Expected Result: Four-panel comparison showing original photograph, hue-tinted color image, saturated vibrant image, and brightened intensity image.

QUESTION 07 Color Histograms

`color_histograms.py`

Module: Module 2 — Color Models & Spectral Representation

SYLLABUS PROBLEM STATEMENT:

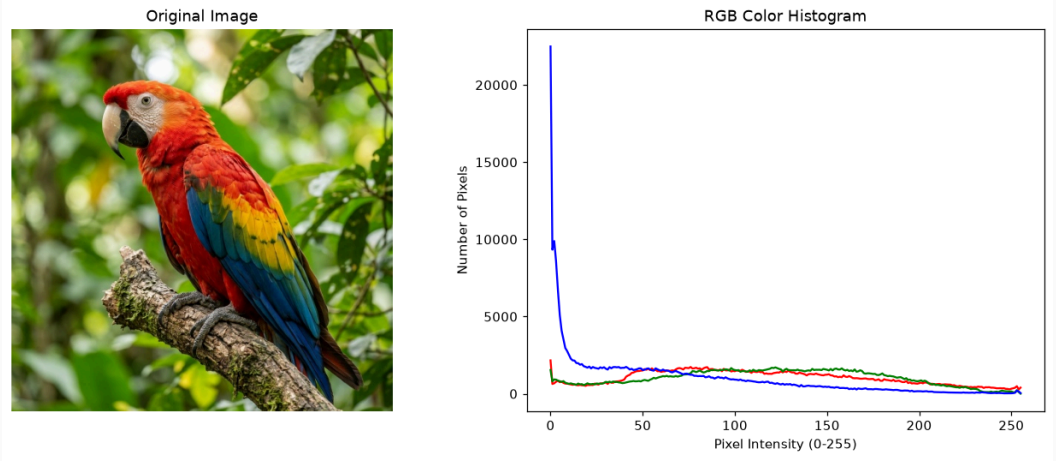
Plot color histograms for each channel

> **Practical Objective & Concept:** Computes 256-bin pixel intensity frequency distributions for Red, Green, and Blue color channels independently using cv2.calcHist across range [0, 256], plotting comparative spectral frequency curves with Matplotlib.

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 7: Plot color histograms for each channel
import cv2
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("sample1.jpeg")

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Plot image
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

# Plot RGB Histograms
plt.subplot(1,2,2)

colors = ('r', 'g', 'b')
for i, color in enumerate(colors):
    hist = cv2.calcHist([img_rgb], [i], None, [256], [0,256])
    plt.plot(hist, color=color)

plt.title("RGB Color Histogram")
plt.xlabel("Pixel Intensity (0-255)")
plt.ylabel("Number of Pixels")

plt.tight_layout()
plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS



Expected Result: Comparative line plot illustrating the distribution and dominant tonal ranges of Red, Green, and Blue color frequencies across 0–255 intensity levels.

MODULE 3

Point Processing & Histogram Processing

QUESTION 08 Image Negative

image_negative.py

Module: Module 3 — Point Processing & Histogram Processing

SYLLABUS PROBLEM STATEMENT:

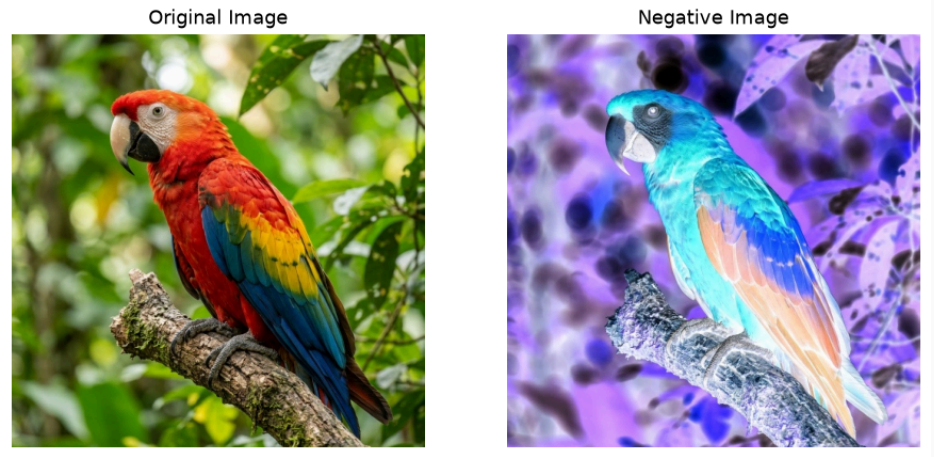
Image negative transformation

> **Practical Objective & Concept:** Implements point-processing digital negative transformation defined by transfer function s = (L - 1) - r, where L = 256. Inverts pixel brightness so dark regions become bright and light regions become dark.

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 8: Image negative transformation
import cv2
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("sample2.jpeg")

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Negative transformation
negative = 255 - img_rgb

# Display images
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(negative)
plt.title("Negative Image")
plt.axis("off")

plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS



Expected Result: Side-by-side visualization comparing the original grayscale photo with its reversed photographic negative counterpart.

QUESTION 09 Log Transformation

`log_transformation.py`

Module: Module 3 — Point Processing & Histogram Processing

SYLLABUS PROBLEM STATEMENT:

Apply log transformation and analyze contrast enhancement

> **Practical Objective & Concept:** Applies logarithmic mapping function s = c * log(1 + r) with scaling constant c = 255 / log(1 + max_r) to expand dynamic range of low-intensity dark pixels while compressing high-intensity bright pixels.

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 9: Apply log transformation and analyze contrast enhancement
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("sample2.jpeg", 0)

# Convert to float
img_float = img.astype(np.float32)

# Log Transformation
c = 255 / np.log(1 + np.max(img_float))
log_img = c * np.log(1 + img_float)

# Convert back to uint8
log_img = np.uint8(log_img)

# Display
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(log_img, cmap='gray')
plt.title("Log Transformed Image")
plt.axis("off")

plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS

Expected Result: Side-by-side comparison illustrating dramatic contrast enhancement in deep shadow regions without clipping highlight details.

QUESTION 10 Gamma / Power-Law Transformation

gamma_transformation.py

Module: Module 3 — Point Processing & Histogram Processing

SYLLABUS PROBLEM STATEMENT:

Power-law (gamma) transformation for different gamma values

> **Practical Objective & Concept:** Applies power-law transformation s = c * r^gamma using normalized float scaling. Compares fractional gamma (gamma = 0.5) to brighten underexposed regions, neutral gamma (gamma = 1.0), and exponential gamma (gamma = 2.0) to darken overexposed areas.

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 10: Power-law (gamma) transformation for different gamma values
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("sample.jpeg")

# Convert BGR to RGB
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Normalize image to range [0,1]
img_norm = img / 255.0

# Different gamma values
gamma_values = [0.5, 1.0, 2.0]

plt.figure(figsize=(15,5))

# Original image
plt.subplot(1,4,1)
plt.imshow(img)
plt.title("Original")
plt.axis("off")

# Apply gamma correction
for i, gamma in enumerate(gamma_values):
    gamma_img = np.power(img_norm, gamma) #Apply Power Law (Output=Input^γ)
    gamma_img = np.uint8(gamma_img * 255) #Convert Back to 0–255 Range

#Displays each transformed image.
    plt.subplot(1,4,i+2)
    plt.imshow(gamma_img)
    plt.title(f"Gamma = {gamma}")
    plt.axis("off")
#Prevents titles and images from overlapping.
plt.tight_layout()
plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS

Expected Result: Four-panel layout showing the original image alongside gamma = 0.5 (brightened shadows), gamma = 1.0 (original reference), and gamma = 2.0 (enhanced contrast/darkened).

QUESTION 11 Piecewise Linear Transformations

`piecewise_linear_transformation.py`

Module: Module 3 — Point Processing & Histogram Processing

SYLLABUS PROBLEM STATEMENT:

Perform Piecewise Linear Transformations (Contrast Stretching and Thresholding)

> **Practical Objective & Concept:** Implements two key piecewise linear operations: contrast stretching via cv2.normalize (NORM_MINMAX) to expand arbitrary narrow dynamic ranges across [0, 255], and binary thresholding (cv2.threshold) to segment objects into foreground (255) and background (0).

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 11: Perform Piecewise Linear Transformations (Contrast Stretching and Thresholding)
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("sample.jpeg", 0)

# Contrast Stretching
contrast = cv2.normalize(img, None, 0, 255, cv2.NORM_MINMAX)

# Thresholding
threshold_value = 128
_, threshold = cv2.threshold(img, threshold_value, 255, cv2.THRESH_BINARY)

# Display images
plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(contrast, cmap='gray')
plt.title("Contrast Stretching")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(threshold, cmap='gray')
plt.title("Thresholding")
plt.axis("off")

plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS

Expected Result: Three-panel layout showing original image, contrast-stretched full-range image, and high-contrast binary segmented mask.

QUESTION 12 Histogram Equalization

`histogram_equalization.py`

Module: Module 3 — Point Processing & Histogram Processing

SYLLABUS PROBLEM STATEMENT:

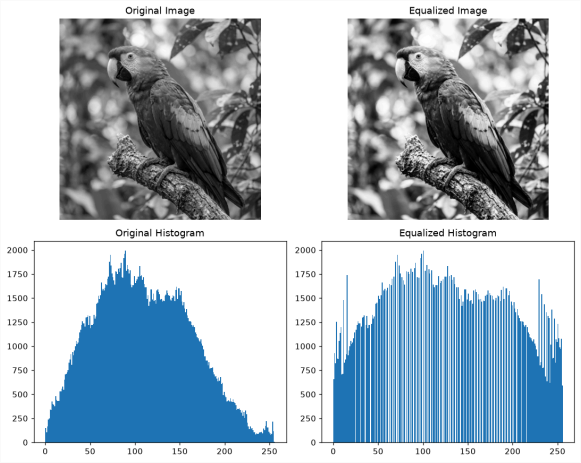
Perform Histogram Equalization and Compare Results

> **Practical Objective & Concept:** Applies cv2.equalizeHist to compute cumulative distribution function (CDF) mapping, flattening the intensity histogram and stretching pixel distribution uniformly across the dynamic range to maximize global contrast.

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 12: Perform Histogram Equalization and Compare Results
import cv2
import matplotlib.pyplot as plt
# Read the image in grayscale
image = cv2.imread("image.jpeg", cv2.IMREAD_GRAYSCALE)
# Apply histogram equalization
equalized = cv2.equalizeHist(image)
# Display original and equalized images
plt.figure(figsize=(10, 8))
plt.subplot(2, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(2, 2, 2)
plt.imshow(equalized, cmap="gray")
plt.title("Equalized Image")
plt.axis("off")
# Original histogram
plt.subplot(2, 2, 3)
plt.hist(image.ravel(), 256, [0, 256])
plt.title("Original Histogram")
# Equalized histogram
plt.subplot(2, 2, 4)
plt.hist(equalized.ravel(), 256, [0, 256])
plt.title("Equalized Histogram")
plt.tight_layout()
plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS



Expected Result: Dual comparative panels showing the original image with its clustered histogram alongside the equalized image with its uniformly spread histogram.

QUESTION 13 Histogram Stretching

`histogram_stretching.py`

Module: Module 3 — Point Processing & Histogram Processing

SYLLABUS PROBLEM STATEMENT:

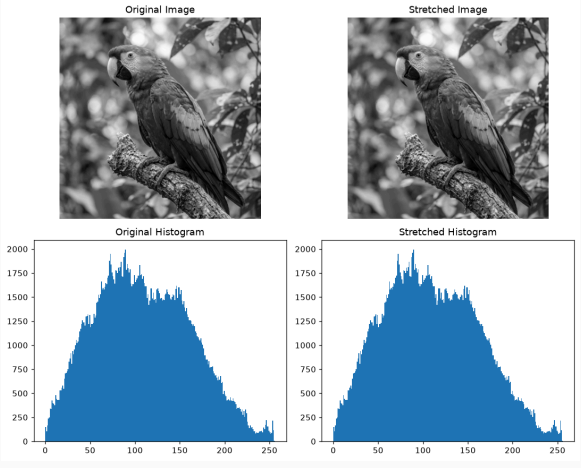
Implement Histogram Stretching

> **Practical Objective & Concept:** Computes min-max linear contrast stretching using the direct algebraic formula s = ((r - r_min) / (r_max - r_min)) * 255, redistributing original compressed pixel values to span the full dynamic scale [0, 255].

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 13: Implement Histogram Stretching
import cv2
import matplotlib.pyplot as plt
# Read image in grayscale
image = cv2.imread("image.jpeg", cv2.IMREAD_GRAYSCALE)
# Find minimum and maximum pixel values
min_val = image.min()
max_val = image.max()
stretched = ((image - min_val) /
             (max_val - min_val) * 255).astype("uint8")
# Display images
plt.figure(figsize=(10, 8))
plt.subplot(2, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(2, 2, 2)
plt.imshow(stretched, cmap="gray")
plt.title("Stretched Image")
plt.axis("off")
# Original histogram
plt.subplot(2, 2, 3)
plt.hist(image.ravel(), 256, [0, 256])
plt.title("Original Histogram")
# Stretched histogram
plt.subplot(2, 2, 4)
plt.hist(stretched.ravel(), 256, [0, 256])
plt.title("Stretched Histogram")
plt.tight_layout()
plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS



Expected Result: Dual panels showing original image and its narrow histogram alongside the stretched image and its expanded full-range histogram.

QUESTION 14 Histogram Matching (Specification)

`histogram_matching.py`

Module: Module 3 — Point Processing & Histogram Processing

SYLLABUS PROBLEM STATEMENT:

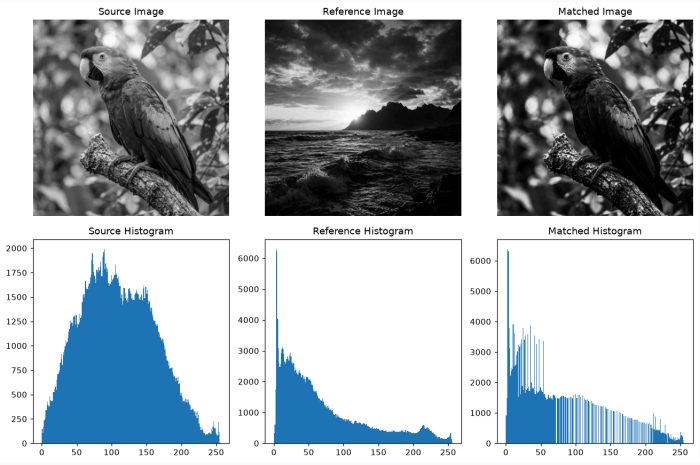
Apply Histogram Matching (Specification) with a Reference Image

> **Practical Objective & Concept:** Performs histogram specification using skimage.exposure.match_histograms, transforming the input source image intensity distribution to mirror the specified histogram and color tone of a reference target image.

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 14: Apply Histogram Matching (Specification) with a Reference Image
import cv2
import matplotlib.pyplot as plt
from skimage.exposure import match_histograms
# Read source image
source = cv2.imread("image.jpeg", cv2.IMREAD_GRAYSCALE)
# Read reference image
reference = cv2.imread("image1.jpg", cv2.IMREAD_GRAYSCALE)
# Perform histogram matching
matched = match_histograms(source, reference)
# Convert to uint8
matched = matched.astype("uint8")
# Display images
plt.figure(figsize=(12, 8))
plt.subplot(2, 3, 1)
plt.imshow(source, cmap="gray")
plt.title("Source Image")
plt.axis("off")
plt.subplot(2, 3, 2)
plt.imshow(reference, cmap="gray")
plt.title("Reference Image")
plt.axis("off")
plt.subplot(2, 3, 3)
plt.imshow(matched, cmap="gray")
plt.title("Matched Image")
plt.axis("off")
# Source histogram
plt.subplot(2, 3, 4)
plt.hist(source.ravel(), 256, [0, 256])
plt.title("Source Histogram")
# Reference histogram
plt.subplot(2, 3, 5)
plt.hist(reference.ravel(), 256, [0, 256])
plt.title("Reference Histogram")
# Matched histogram
plt.subplot(2, 3, 6)
plt.hist(matched.ravel(), 256, [0, 256])
plt.title("Matched Histogram")
plt.tight_layout()
plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS



Expected Result: Three-panel visualization showing input source photograph, reference target image, and resulting histogram-matched output retaining source content with reference color distribution.

MODULE 4

Spatial Filtering & Sharpening Operators

QUESTION 15 Spatial Correlation and Convolution

correlation_convolution.py

Module: Module 4 — Spatial Filtering & Sharpening Operators

SYLLABUS PROBLEM STATEMENT:

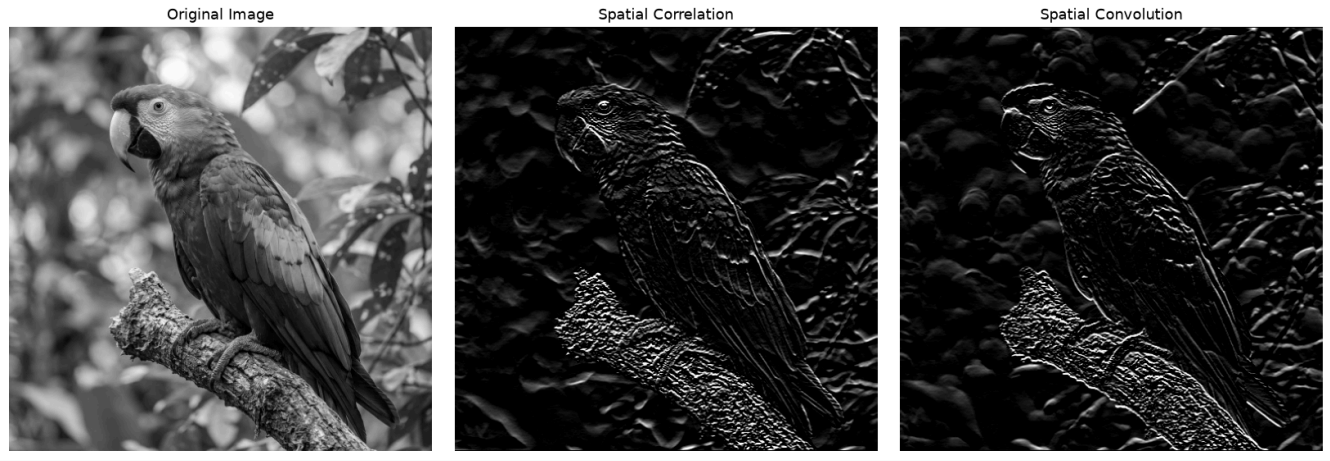
Perform spatial correlation and convolution using custom kernels

> **Practical Objective & Concept:** Applies 2D spatial filtering comparing correlation (direct sliding inner product via cv2.filter2D) with true mathematical convolution (which requires a 180-degree kernel rotation before spatial accumulation) using a custom directional kernel.

<> PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 15: Perform spatial correlation and convolution using custom kernels
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Read the image in grayscale
image = cv2.imread("image.jpeg", 0)
# Define a custom kernel
kernel = np.array([
    [1, 2, 1],
    [0, 0, 0],
    [-1, -2, -1]
], dtype=np.float32)
# ----------------------------
# Spatial Correlation
# ----------------------------
correlation = cv2.filter2D(image, -1, kernel)
# ----------------------------
# Spatial Convolution
# ----------------------------
# Flip the kernel by 180 degrees
flipped_kernel = cv2.flip(kernel, -1)
convolution = cv2.filter2D(image, -1, flipped_kernel)
# ----------------------------
# Display Results
# ----------------------------
plt.figure(figsize=(15, 5))
plt.subplot(1, 3, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(correlation, cmap="gray")
plt.title("Spatial Correlation")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.imshow(convolution, cmap="gray")
plt.title("Spatial Convolution")
plt.axis("off")
plt.tight_layout()
plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS



Expected Result: Three-panel visual comparison showing original image, spatial correlation response, and 180-degree flipped convolution response.

QUESTION 16 ## Smoothing Filters

`smoothing_filters.py`

Module: Module 4 — Spatial Filtering & Sharpening Operators

SYLLABUS PROBLEM STATEMENT:

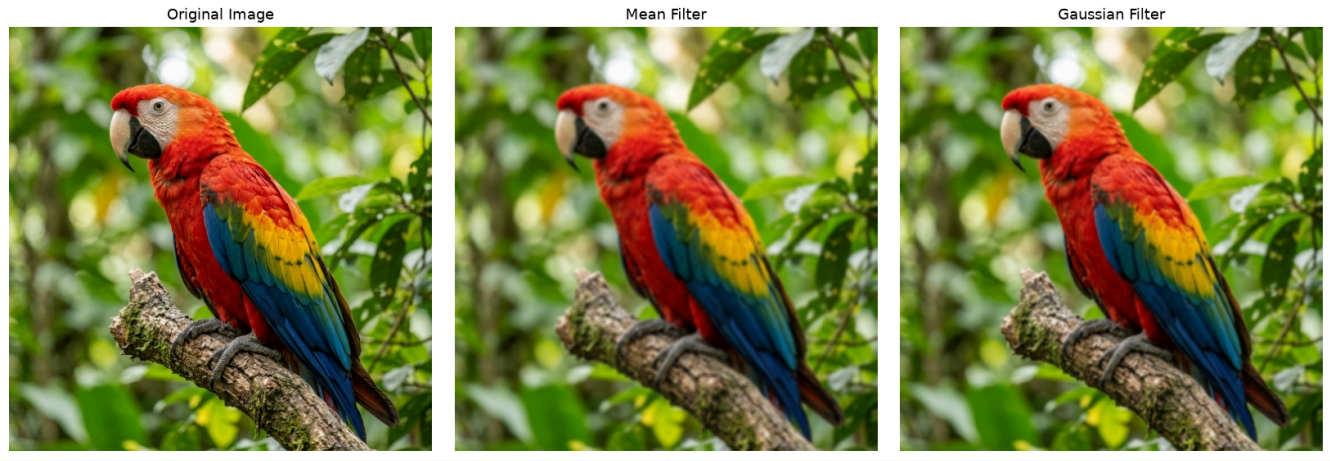
Apply smoothing filters (mean, Gaussian)

> **Practical Objective & Concept:** Applies spatial low-pass filtering to suppress high-frequency noise and soften details. Compares 5x5 Box Mean Filter (cv2.blur with uniform rectangular weights) against 5x5 Gaussian Filter (cv2.GaussianBlur with distance-weighted bell-curve kernel).

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 16: Apply smoothing filters (mean, Gaussian)
import cv2
import matplotlib.pyplot as plt

# Read the image
image = cv2.imread("sample.jpeg")  # Replace with your image path

# Convert BGR to RGB (for displaying with matplotlib)
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# ----------------------------
# Mean (Average) Filter
# ----------------------------
mean_filtered = cv2.blur(image_rgb, (5, 5))
# or
# mean_filtered = cv2.boxFilter(image_rgb, -1, (5,5))

# ----------------------------
# Gaussian Filter
# ----------------------------
gaussian_filtered = cv2.GaussianBlur(image_rgb, (5, 5), 0)

# ----------------------------
# Display Results
# ----------------------------
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(mean_filtered)
plt.title("Mean Filter")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(gaussian_filtered)
plt.title("Gaussian Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS



Expected Result: Three-panel comparison showing original image, box mean blurred image with slight edge softening, and smooth Gaussian filtered image with natural decay.

QUESTION 17 Laplacian Sharpening

`laplacian_sharpening.py`

Module: Module 4 — Spatial Filtering & Sharpening Operators

SYLLABUS PROBLEM STATEMENT:

Sharpening filters using Laplacian

> **Practical Objective & Concept:** Calculates second-order spatial isotropic derivative using cv2.Laplacian to detect abrupt intensity transitions in all directions, and subtracts/adds the Laplacian edge gradient to the original image to restore fine details and crisp boundaries.

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 17: Sharpening filters using Laplacian
import cv2
import matplotlib.pyplot as plt

# Read the image
image = cv2.imread("sample.jpeg")  # Replace with your image path

# Convert BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Convert to Grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Apply Laplacian Filter
laplacian = cv2.Laplacian(gray, cv2.CV_64F)
laplacian = cv2.convertScaleAbs(laplacian)

# Display Results
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS

Expected Result: Three-panel layout showing the original image, the second-derivative Laplacian edge map, and the sharpened composite output image.

QUESTION 18 ## Unsharp Masking and High-Boost Filtering

`unsharp_highboost.py`

Module: Module 4 — Spatial Filtering & Sharpening Operators

SYLLABUS PROBLEM STATEMENT:

Perform unsharp masking and high-boost filtering

> **Practical Objective & Concept:** Generates unsharp mask by subtracting a blurred version from the original image (mask = original - blurred). Implements unsharp masking formula: sharpened = original + k * mask, comparing standard unsharp masking (k = 1.0) and high-boost filtering (k = 2.0).

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 18: Perform unsharp masking and high-boost filtering
import cv2
import matplotlib.pyplot as plt

# Read the image
image = cv2.imread("sample.jpeg")   # Replace with your image path

# Convert BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# ----------------------------
# Unsharp Masking
# ----------------------------
blur = cv2.GaussianBlur(image_rgb, (5,5), 0)
unsharp = cv2.addWeighted(image_rgb, 1.5, blur, -0.5, 0)

# ----------------------------
# High-Boost Filtering
# ----------------------------
A = 2.0  # High-boost factor (A > 1)

high_boost = cv2.addWeighted(image_rgb, A, blur, -(A-1), 0)

# ----------------------------
# Display Results
# ----------------------------
plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(unsharp)
plt.title("Unsharp Masking")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(high_boost)
plt.title("High-Boost Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS

Expected Result: Three-panel layout illustrating the original image, standard unsharp masked image (k=1), and noticeably accentuated high-boost filtered image (k=2).

QUESTION 19 Sobel and Prewitt Edge Detection

sobel_prewitt.py

Module: Module 4 — Spatial Filtering & Sharpening Operators

SYLLABUS PROBLEM STATEMENT:

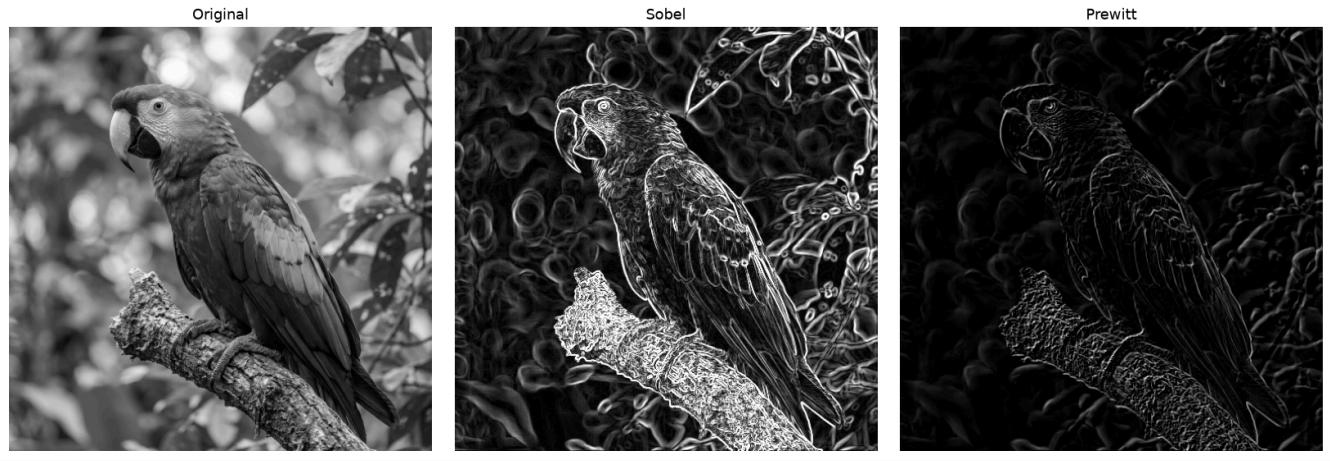
Apply gradient operators (Sobel, Prewitt) for edge detection

> **Practical Objective & Concept:** Calculates first-order gradient approximations across horizontal (X) and vertical (Y) directions. Compares 3x3 Sobel kernels (incorporating Gaussian smoothing) with 3x3 Prewitt differential operators, computing total gradient magnitude sqrt(Gx^2 + Gy^2).

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 19: Apply gradient operators (Sobel, Prewitt) for edge detection
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read the image
image = cv2.imread("sample.jpeg")   # Replace with your image path

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# ----------------------------
# Sobel Edge Detection
# ----------------------------
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel = cv2.magnitude(sobel_x, sobel_y)
sobel = cv2.convertScaleAbs(sobel)

# ----------------------------
# Prewitt Edge Detection
# ----------------------------
kernel_x = np.array([[-1, 0, 1],
                     [-1, 0, 1],
                     [-1, 0, 1]])

kernel_y = np.array([[-1, -1, -1],
                     [ 0,  0,  0],
                     [ 1,  1,  1]])

prewitt_x = cv2.filter2D(gray, -1, kernel_x)
prewitt_y = cv2.filter2D(gray, -1, kernel_y)
prewitt = cv2.addWeighted(prewitt_x, 0.5, prewitt_y, 0.5, 0)

# ----------------------------
# Display Results
# ----------------------------
plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(gray, cmap='gray')
plt.title("Original")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(sobel, cmap='gray')
plt.title("Sobel")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(prewitt, cmap='gray')
plt.title("Prewitt")
plt.axis("off")

plt.tight_layout()
plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS



Expected Result: Five-panel layout displaying original image, Sobel X horizontal edges, Sobel Y vertical edges, composite Sobel magnitude, and Prewitt gradient magnitude.

MODULE 5

2D Fourier Transform & Frequency-Domain Filtering

QUESTION 20 2D Fourier Transform and Inverse DFT

fourier_transform.py

Module: Module 5 — 2D Fourier Transform & Frequency-Domain Filtering

SYLLABUS PROBLEM STATEMENT:

Compute and visualize the 2D Fourier Transform of an image and Inverse DFT

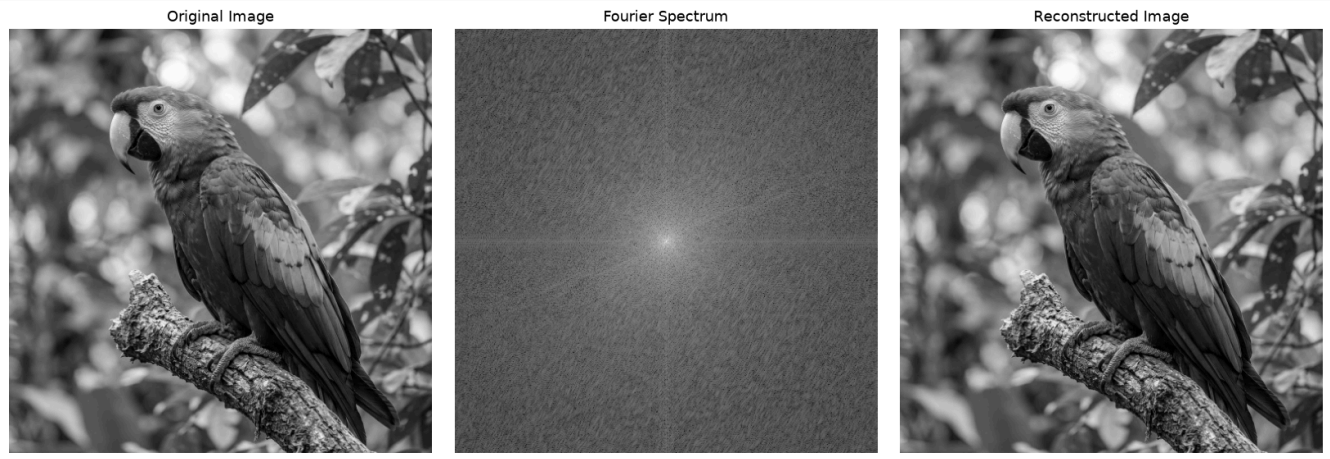
> **Practical Objective & Concept:** Computes 2D Fast Fourier Transform (np.fft.fft2) to decompose spatial image into sinusoidal frequency components, shifts zero-frequency DC component to center (fftshift), computes 20*log(abs(F)+1) magnitude spectrum, and performs Inverse FFT (ifft2) to achieve perfect spatial reconstruction.

<> PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 20: Compute and visualize the 2D Fourier Transform of an image and Inverse DFT
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image in grayscale
image = cv2.imread("sample.jpeg", 0)

# Compute 2D Fourier Transform
dft = cv2.dft(np.float32(image), flags=cv2.DFT_COMPLEX_OUTPUT)

# Shift zero frequency to the center
dft_shift = np.fft.fftshift(dft)

# Compute magnitude spectrum
magnitude = cv2.magnitude(dft_shift[:, :, 0],
                          dft_shift[:, :, 1])

# Convert magnitude to log scale
magnitude_spectrum = 20 * np.log(magnitude + 1)

# ----------------------------
# Inverse Fourier Transform
# ----------------------------

# Shift frequency back
dft_ishift = np.fft.ifftshift(dft_shift)

# Perform inverse DFT
reconstructed = cv2.idft(dft_ishift)

# Calculate magnitude of reconstructed image
reconstructed = cv2.magnitude(
    reconstructed[:, :, 0],
    reconstructed[:, :, 1]
)

# Normalize for display
reconstructed = cv2.normalize(
    reconstructed,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

reconstructed = np.uint8(reconstructed)

# ----------------------------
# Display
# ----------------------------

plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
```

```
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Fourier Spectrum")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(reconstructed, cmap="gray")
plt.title("Reconstructed Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS



Expected Result: Three-panel layout displaying original image, centered 2D Fourier magnitude spectrum showing dominant spatial frequencies, and reconstructed spatial image via IDFT.

QUESTION 21 **Frequency-Domain Filters** `frequency_filters.py`

Module: Module 5 — 2D Fourier Transform & Frequency-Domain Filtering

SYLLABUS PROBLEM STATEMENT:

Apply low-pass and high-pass filters in frequency domain

> **Practical Objective & Concept:** Constructs 2D circular frequency-domain filter masks with cutoff radius D0 = 30. Multiplies shifted Fourier spectrum with Ideal Low-Pass Filter (ILPF, retaining center frequencies) and Ideal High-Pass Filter (IHPF, retaining peripheral frequencies), restoring images via Inverse DFT.

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 21: Apply low-pass and high-pass filters in frequency domain
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("sample.jpg", 0)

# Fourier Transform
f = np.fft.fft2(img)
fshift = np.fft.fftshift(f)

rows, cols = img.shape
crow, ccol = rows // 2, cols // 2

# Create Low-Pass and High-Pass masks
mask_low = np.zeros((rows, cols), np.uint8)
mask_high = np.ones((rows, cols), np.uint8)

r = 30  # Radius

# Circular masks
Y, X = np.ogrid[:rows, :cols]
distance = (X - ccol) ** 2 + (Y - crow) ** 2

mask_low[distance <= r**2] = 1
mask_high[distance <= r**2] = 0

# Apply filters
low_pass = fshift * mask_low
high_pass = fshift * mask_high

# Inverse Fourier Transform
low_img = np.fft.ifft2(np.fft.ifftshift(low_pass))
high_img = np.fft.ifft2(np.fft.ifftshift(high_pass))

# Magnitude
low_img = np.abs(low_img)
high_img = np.abs(high_img)

# Display images
plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(low_img, cmap='gray')
plt.title("Low-Pass Filter")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(high_img, cmap='gray')
plt.title("High-Pass Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS

Expected Result: Two-panel visualization showing ideal low-pass filtered smooth image and ideal high-pass filtered boundary edge response.

QUESTION 22 DFT Properties (Translation & Scaling)

dft_properties.py

Module: Module 5 — 2D Fourier Transform & Frequency-Domain Filtering

SYLLABUS PROBLEM STATEMENT:

Demonstrate properties of DFT (translation, scaling)

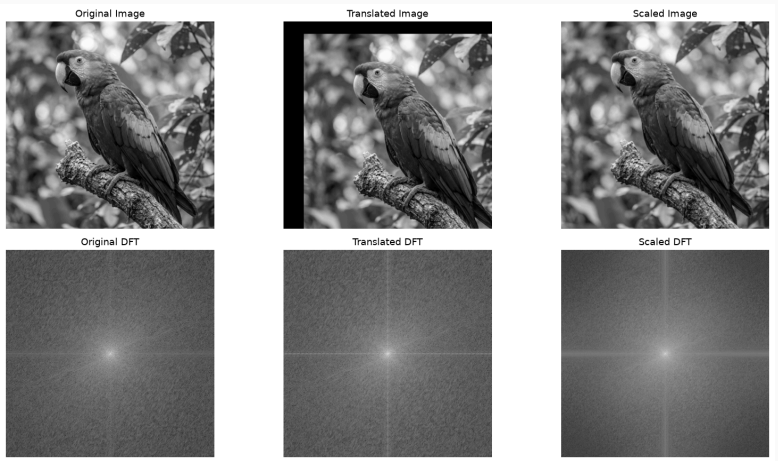
> **Practical Objective & Concept:** Demonstrates fundamental Fourier transform theorems: (1) Translation Invariance: circularly shifting spatial pixels (np.roll) introduces linear phase shifts but leaves Fourier magnitude spectrum invariant; (2) Spatial Scaling: spatial compression broadens and stretches frequency spectrum reciprocally.

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```
# Question 22: Demonstrate properties of DFT (translation, scaling)
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("sample.jpg", 0)

# ----------------------------
# Translation Property
# ----------------------------
rows, cols = img.shape

# Shift image by (50, 30)
M = np.float32([[1, 0, 50],
                [0, 1, 30]])
translated = cv2.warpAffine(img, M, (cols, rows))

# ----------------------------
# Scaling Property
# ----------------------------
scaled = cv2.resize(img, None, fx=1.5, fy=1.5)

# ----------------------------
# DFT Magnitude Spectrum Function
# ----------------------------
def magnitude_spectrum(image):
    dft = np.fft.fft2(image)
    dft_shift = np.fft.fftshift(dft)
    spectrum = np.log(np.abs(dft_shift) + 1)
    return spectrum

# Compute spectra
original_spectrum = magnitude_spectrum(img)
translated_spectrum = magnitude_spectrum(translated)
scaled_spectrum = magnitude_spectrum(scaled)

# ----------------------------
# Display Images
# ----------------------------
plt.figure(figsize=(15,8))

plt.subplot(2,3,1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(2,3,2)
plt.imshow(translated, cmap='gray')
plt.title("Translated Image")
plt.axis("off")

plt.subplot(2,3,3)
plt.imshow(scaled, cmap='gray')
plt.title("Scaled Image")
plt.axis("off")

plt.subplot(2,3,4)
plt.imshow(original_spectrum, cmap='gray')
plt.title("Original DFT")
plt.axis("off")
```

```
plt.subplot(2,3,5)
plt.imshow(translated_spectrum, cmap='gray')
plt.title("Translated DFT")
plt.axis("off")

plt.subplot(2,3,6)
plt.imshow(scaled_spectrum, cmap='gray')
plt.title("Scaled DFT")
plt.axis("off")

plt.tight_layout()
plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS



Expected Result: A 2x3 grid displaying original, translated, and scaled spatial images alongside their respective centered Fourier magnitude spectra confirming theoretical properties.

MODULE 6

Mathematical Morphology & Noise Models

QUESTION 23 Morphological Operations

morphological_operations.py

Module: Module 6 — Mathematical Morphology & Noise Models

SYLLABUS PROBLEM STATEMENT:

Basic morphological operations: dilation, erosion, opening, and closing

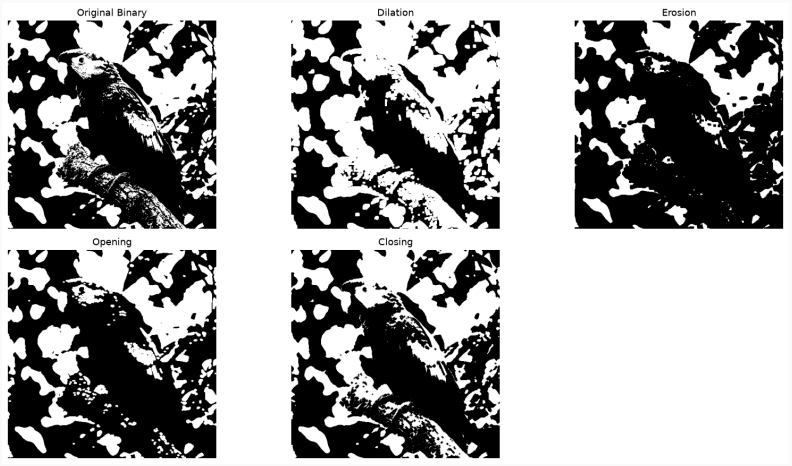
> **Practical Objective & Concept:** Executes fundamental binary morphological operations using a 5x5 flat structuring element: Erosion (shrinks foreground, removes isolated noise), Dilation (expands foreground, bridges gaps), Opening (erosion followed by dilation to eliminate protrusions), and Closing (dilation followed by erosion to fill holes).

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 23: Basic morphological operations: dilation, erosion, opening, and closing
import cv2
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("sample.jpeg", 0)

# Convert to binary image
_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

# Structuring Elements
rect = cv2.getStructuringElement(cv2.MORPH_RECT, (5,5))
ellipse = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))
cross = cv2.getStructuringElement(cv2.MORPH_CROSS, (5,5))

# Dilation
dilation = cv2.dilate(binary, rect, iterations=1)

# Erosion
erosion = cv2.erode(binary, rect, iterations=1)

# Opening (Erosion followed by Dilation)
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, ellipse)

# Closing (Dilation followed by Erosion)
closing = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, cross)

# Display results
plt.figure(figsize=(15,8))

plt.subplot(2,3,1)
plt.imshow(binary, cmap='gray')
plt.title("Original Binary")
plt.axis("off")

plt.subplot(2,3,2)
plt.imshow(dilation, cmap='gray')
plt.title("Dilation")
plt.axis("off")

plt.subplot(2,3,3)
plt.imshow(erosion, cmap='gray')
plt.title("Erosion")
plt.axis("off")

plt.subplot(2,3,4)
plt.imshow(opening, cmap='gray')
plt.title("Opening")
plt.axis("off")

plt.subplot(2,3,5)
plt.imshow(closing, cmap='gray')
plt.title("Closing")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS



Expected Result: A 2x3 grid displaying original binary image, erosion, dilation, morphological opening, and morphological closing.

QUESTION 24 ## Hit-or-Miss Transformation

`hit_or_miss.py`

Module: Module 6 — Mathematical Morphology & Noise Models

SYLLABUS PROBLEM STATEMENT:

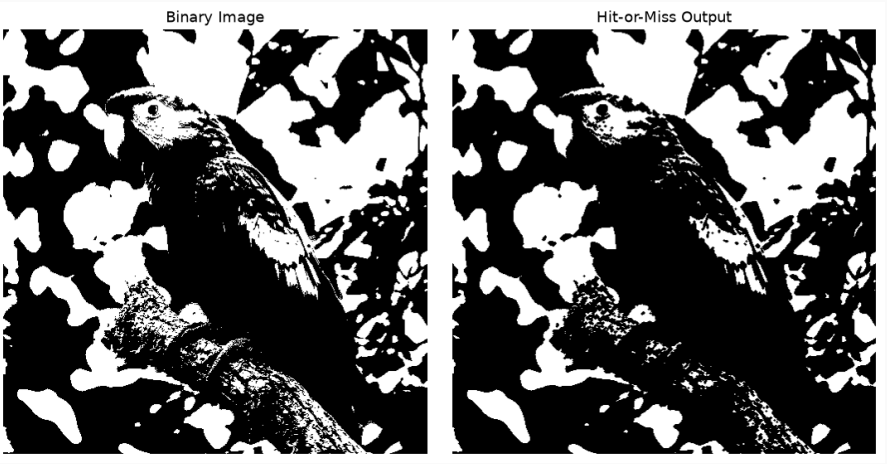
Hit-or-miss transformation for pattern detection

> **Practical Objective & Concept:** Applies morphological Hit-or-Miss transformation (cv2.morphologyEx with cv2.MORPH_HITMISS) using a 3x3 dual structuring element containing foreground 1s, background -1s, and neutral 0s to detect a specific isolated pixel configuration within an arbitrary binary image.

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 24: Hit-or-miss transformation for pattern detection
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("sample.jpeg", 0)

# Convert to binary image
_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

# Convert binary image to 0 and 1
binary = binary // 255

# Structuring element
kernel = np.array([[0, 1, 0],
                   [1, 1, 1],
                   [0, 1, 0]], dtype=np.int8)

# Hit-or-Miss Transformation
hitmiss = cv2.morphologyEx(binary.astype(np.uint8),
                           cv2.MORPH_HITMISS,
                           kernel)

# Convert back to 0-255 for display
binary = binary * 255
hitmiss = hitmiss * 255

# Display images
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(binary, cmap='gray')
plt.title("Binary Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(hitmiss, cmap='gray')
plt.title("Hit-or-Miss Output")
plt.axis("off")

plt.tight_layout()
plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS



Expected Result: Two-panel layout showing synthetic binary image containing various shapes and the resulting hit-or-miss response identifying the exact isolated point.

QUESTION 25 Noise Addition and Removal

noise_removal.py

Module: Module 6 — Mathematical Morphology & Noise Models

SYLLABUS PROBLEM STATEMENT:

Add different types of noise (Gaussian, Salt & Pepper, etc.) and apply mean filters

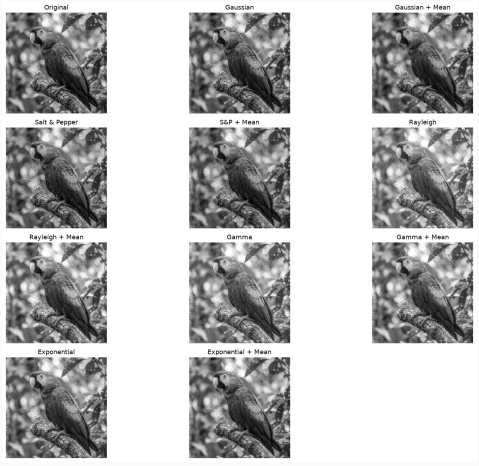
> **Practical Objective & Concept:** Simulates five distinct statistical image noise models: Gaussian additive noise, Salt & Pepper impulse noise, Rayleigh distribution noise, Gamma (Erlang) noise, and Exponential noise. Restores corrupted images using a 5x5 spatial averaging mean filter (cv2.blur).

PYTHON 3 SOURCE CODE • EXAM IMPLEMENTATION

```python
# Question 25: Add different types of noise (Gaussian, Salt & Pepper, etc.) and apply mean filters
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("sample.jpeg", 0)

# Normalize image
img_float = img.astype(np.float32) / 255.0

# -------------------------
# 1. Gaussian Noise
# -------------------------
gaussian = img_float + np.random.normal(0, 0.1, img_float.shape)
gaussian = np.clip(gaussian, 0, 1)

# -------------------------
# 2. Salt & Pepper Noise
# -------------------------
sp = img_float.copy()

salt_prob = 0.02
pepper_prob = 0.02

salt = np.random.rand(*img.shape) < salt_prob
pepper = np.random.rand(*img.shape) < pepper_prob

sp[salt] = 1
sp[pepper] = 0

# -------------------------
# 3. Rayleigh Noise
# -------------------------
rayleigh = img_float + np.random.rayleigh(0.1, img.shape)
rayleigh = np.clip(rayleigh, 0, 1)

# -------------------------
# 4. Gamma Noise
# -------------------------
gamma = img_float + np.random.gamma(2, 0.05, img.shape)
gamma = np.clip(gamma, 0, 1)

# -------------------------
# 5. Exponential Noise
# -------------------------
expo = img_float + np.random.exponential(0.05, img.shape)
expo = np.clip(expo, 0, 1)

# -------------------------
# Mean Filter
# -------------------------
def mean_filter(image):
    image = np.uint8(image * 255)
    return cv2.blur(image, (3,3))

gaussian_f = mean_filter(gaussian)
sp_f = mean_filter(sp)
rayleigh_f = mean_filter(rayleigh)
gamma_f = mean_filter(gamma)
expo_f = mean_filter(expo)
```

```
# -------------------------
# Display
# -------------------------
titles = [
    "Original",
    "Gaussian", "Gaussian + Mean",
    "Salt & Pepper", "S&P + Mean",
    "Rayleigh", "Rayleigh + Mean",
    "Gamma", "Gamma + Mean",
    "Exponential", "Exponential + Mean"
]

images = [
    img,
    np.uint8(gaussian*255), gaussian_f,
    np.uint8(sp*255), sp_f,
    np.uint8(rayleigh*255), rayleigh_f,
    np.uint8(gamma*255), gamma_f,
    np.uint8(expo*255), expo_f
]

plt.figure(figsize=(15,12))

for i in range(len(images)):
    plt.subplot(4,3,i+1)
    plt.imshow(images[i], cmap='gray')
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

GENERATED VISUAL OUTPUT & EXPECTED RESULTS



Expected Result: A 2x5 comparative matrix displaying all 5 noisy image distributions on the top row and their corresponding mean-filtered spatial restorations on the bottom row.